

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vt29748@veltech.edu.in

Roll no: vt29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 6_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 44.5

Section 1 : Coding

1. Problem Statement

Virat is given a group of individuals, and he wants to identify if there is an influential individual within the group. An influential individual is defined as someone who is followed by more people than they follow.

He is provided with a list of follow-up relationships among the individuals in the form of a matrix. Your task is to help Virat determine if there is an influential individual and, if so, find their index within the group.

Write a program to solve this problem using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of individuals in the group.

The following N lines will contain N space-separated integers, representing the following relationship matrix for the individuals.

Output Format

If an influential individual is found, print "Influential Individual: X," where X is the index (0-based) of the influential individual.

Otherwise, print "There is no influential individual in the group".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 1 1 0

0 0 1 0

0 0 0 0

0 0 1 0

Output: Influential Individual: 2

Answer

```
#include <stdio.h>
```

```
void findInfluentialIndividual() {  
    int n;  
    if (scanf("%d", &n) != 1) return;
```

```
    int matrix[10][10];
```

```
    // Input the adjacency matrix  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%d", &matrix[i][j]);  
        }  
    }  
}
```

```
    // Warshall's Algorithm to find Transitive Closure  
    for (int k = 0; k < n; k++) {  
        for (int i = 0; i < n; i++) {
```

```

        for (int j = 0; j < n; j++) {
            if (matrix[i][k] && matrix[k][j]) {
                matrix[i][j] = 1;
            }
        }
    }
}

int found = 0;
for (int i = 0; i < n; i++) {
    int followsMe = 0;
    int iFollow = 0;

    for (int j = 0; j < n; j++) {
        // Self-loops are usually ignored in influence count
        if (i == j) continue;

        if (matrix[j][i] == 1) followsMe++;
        if (matrix[i][j] == 1) iFollow++;
    }

    if (followsMe > iFollow) {
        printf("Influential Individual: %d\n", i);
        found = 1;
        break; // Stop at the first influential individual found
    }
}

if (!found) {
    printf("There is no influential individual in the group\n");
}

int main() {
    findInfluentialIndividual();
    return 0;
}

```

Status : Partially correct

Marks : 4.5/10

2. Problem statement

Sarah has been tasked with developing a program to determine the shortest paths among nodes in a directed network represented as a graph containing nodes and weighted edges.

Her goal is to implement the Floyd-Warshall algorithm to compute the shortest path matrix for this directed network.

Input Format

The first line of input consists of an integer n , representing the number of nodes in the network.

The next n lines contain a space-separated integer, representing the adjacency matrix 'graph' of the network. Each integer is the weight of the edge from node i to node j .

Output Format

The output prints the shortest path matrix as ' $n \times n$ ' integers, where each element $\text{graph}[i][j]$ represents the shortest distance from node ' i ' to node ' j '.

Print each row of the matrix on a new line, with elements separated by a space, and if no path exists, 999 will be displayed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 3 999 4

8 0 2 999

5 999 0 1

2 999 999 0

Output: 0 3 5 4

5 0 2 3

3 6 0 1

2 5 7 0

Answer

```
#include <stdio.h>
```

```
#define INF 999
```

```
void floydWarshall(int n, int graph[100][100]) {  
    int i, j, k;
```

```
    for (k = 0; k < n; k++) {  
        for (i = 0; i < n; i++) {  
            for (j = 0; j < n; j++) {  
                if (graph[i][k] + graph[k][j] < graph[i][j]) {  
                    graph[i][j] = graph[i][k] + graph[k][j];  
                }  
            }  
        }  
    }
```

```
    for (i = 0; i < n; i++) {  
        for (j = 0; j < n; j++) {  
            printf("%d ", graph[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
int main() {  
    int n;  
    int graph[100][100];  
  
    if (scanf("%d", &n) != 1) return 0;
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%d", &graph[i][j]);  
        }  
    }
```

```
    floydWarshall(n, graph);
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

You are working as a backend engineer for a drone delivery logistics company that operates across multiple cities. Each city is represented as a node, and the drone flight paths between cities are represented as directed edges with weights indicating the time (in minutes) it takes for a drone to travel between them.

Due to varying air traffic, regulations, and geography, the direct travel time between cities may not always be the fastest. Sometimes, going through intermediate cities results in shorter delivery times.

Your task is to build a system that calculates the shortest possible delivery time between every pair of cities, considering all possible intermediate routes.

To efficiently compute this, you're required to implement the Floyd-Warshall Algorithm.

Input Format

The first line of input contains an integer N representing the total number of cities.

The next N lines contains N space-separated integers, representing the delivery time matrix:

$\text{time}[i][j]$ is the time (in minutes) taken to fly directly from City i to City j .

A value of 0 means the same city (i.e., $i == j$).

A value of 999 indicates no direct route between City i and City j .

Output Format

The output prints the shortest distance between every pair of cities.

If there is no shortest path between two cities, then print INF.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

Output: 0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Answer

```
#include <stdio.h>
```

```
#define MAX_CITIES 100
```

```
#define NO_ROUTE 999
```

```
void calculateShortestDeliveryTimes() {  
    int n;  
    int dist[MAX_CITIES][MAX_CITIES];
```

```
    if (scanf("%d", &n) != 1) return;
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%d", &dist[i][j]);  
        }  
    }
```

```
    for (int k = 0; k < n; k++) {  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                if (dist[i][k] != NO_ROUTE && dist[k][j] != NO_ROUTE) {  
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {  
                        dist[i][j] = dist[i][k] + dist[k][j];  
                    }  
                }  
            }  
        }  
    }
```

```

    }
    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (dist[i][j] == NO_ROUTE) {
            printf("INF ");
        } else {
            printf("%d ", dist[i][j]);
        }
    }
    printf("\n");
}

int main() {
    calculateShortestDeliveryTimes();
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays, $val[0..n-1]$ and $wt[0..n-1]$ which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of $val[]$ such that the sum of the weights of this subset is smaller than or equal to W . You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

Input Format

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

Output Format

The output prints the maximum value of items that can be held by the knapsack.

Sample Test Case

Input: 3

60 100 120

10 20 30

50

Output: 220

Answer

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int n, W;  
    int val[101], wt[101];  
  
    if (scanf("%d", &n) != 1) return 0;
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &val[i]);  
    }
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &wt[i]);  
    }
```

```
    scanf("%d", &W);
```

```
    int dp[n + 1][W + 1];
```

```

for (int i = 0; i <= n; i++) {
    for (int w = 0; w <= W; w++) {
        if (i == 0 || w == 0) {
            dp[i][w] = 0;
        } else if (wt[i - 1] <= w) {
            dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
        } else {
            dp[i][w] = dp[i - 1][w];
        }
    }
}

printf("%d\n", dp[n][W]);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Sarah loves challenges and has recently taken up a new programming problem. She is fascinated by the classic "0/1 Knapsack Problem" and wants to create a program that can efficiently solve it.

The 0/1 Knapsack Problem involves selecting a combination of items with given weights and values to maximize the total value, while not exceeding a given weight capacity.

Write a program that takes input for the 0/1 Knapsack Problem and outputs the maximum value that can be achieved, as well as the selected weights that contribute to this maximum value.

Note: The weights are printed in reverse order of the input, meaning the algorithm starts checking from the last item and prints items that are included first.

Input Format

The first line contains an integer n , representing the number of items available.

The second line contains n integers separated by space, representing the values of the items. The i th integer corresponds to the value of the i th item.

The third line contains n integers separated by space, representing the weights of the items. The i th integer corresponds to the weight of the i th item.

The fourth line contains an integer W , representing the maximum weight capacity of the knapsack.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, representing the selected weights contributing to the maximum value.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: Maximum value: 220
Selected weights: 30 20

Answer

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int n, W;  
    int val[11], wt[11];
```

```
if (scanf("%d", &n) != 1) return 0;
```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &val[i]);  
}
```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &wt[i]);  
}
```

```
scanf("%d", &W);
```

```
int dp[12][1001];
```

```
for (int i = 0; i <= n; i++) {  
    for (int w = 0; w <= W; w++) {  
        if (i == 0 || w == 0) {  
            dp[i][w] = 0;  
        } else if (wt[i - 1] <= w) {  
            dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);  
        } else {  
            dp[i][w] = dp[i - 1][w];  
        }  
    }  
}
```

```
printf("Maximum value: %d\n", dp[n][W]);  
printf("Selected weights: ");
```

```
int res = dp[n][W];  
int w = W;  
for (int i = n; i > 0 && res > 0; i--) {  
    if (res != dp[i - 1][w]) {  
        printf("%d ", wt[i - 1]);  
        res = res - val[i - 1];  
        w = w - wt[i - 1];  
    }  
}  
printf("\n");  
return 0;
```

}

Status : Correct

Marks : 10/10